

UDP Client Experiment

UDP Client Experiment

- 1. UDP Overview
- 2. Core Concepts
 - 2.1 UDP Socket Communication Flow
 - 2.2 `sendto()` and `recvfrom()`
 - 2.3 Non-Blocking Receive (MSG_DONTWAIT)
 - 2.4 Network Byte Order
- 3. Project Architecture and Flow
 - 3.1 UDP Client
- 4. Hardware Dependencies
- 5. Source Code Analysis — UDP Client
 - 5.1 Macro Configuration
 - 5.2 WiFi Initialization (Review)
 - 5.3 `vUdpClientTask()` — Send/Receive Loop
- 6. Operation Flow
 - 6.1 Compilation and Flashing Flow
 - 6.2 Test Method
 - 7.3 Normal Log — Client
- 8. Common Problems

1. UDP Overview

UDP (User Datagram Protocol) is a **connectionless, unreliable but efficient** transport layer protocol. Unlike TCP, UDP doesn't establish connections, doesn't guarantee delivery, doesn't guarantee order, but **low overhead, low latency**, very suitable for scenarios with high real-time requirements and tolerable occasional packet loss.

In IoT applications, UDP is widely used for **high-frequency sensor data reporting** (e.g., temperature/humidity sampled 100 times per second, losing a few frames is fine), **LAN device discovery** (broadcast/multicast to detect online devices in same subnet), **DNS domain name resolution** (small query volume, timeout retry works), and **real-time audio/video streaming** (latency priority over画质完整性). Mastering UDP's `sendto()` / `recvfrom()` programming model is the foundation for advanced experiments like custom protocol packing, real-time data reporting and LAN device discovery.

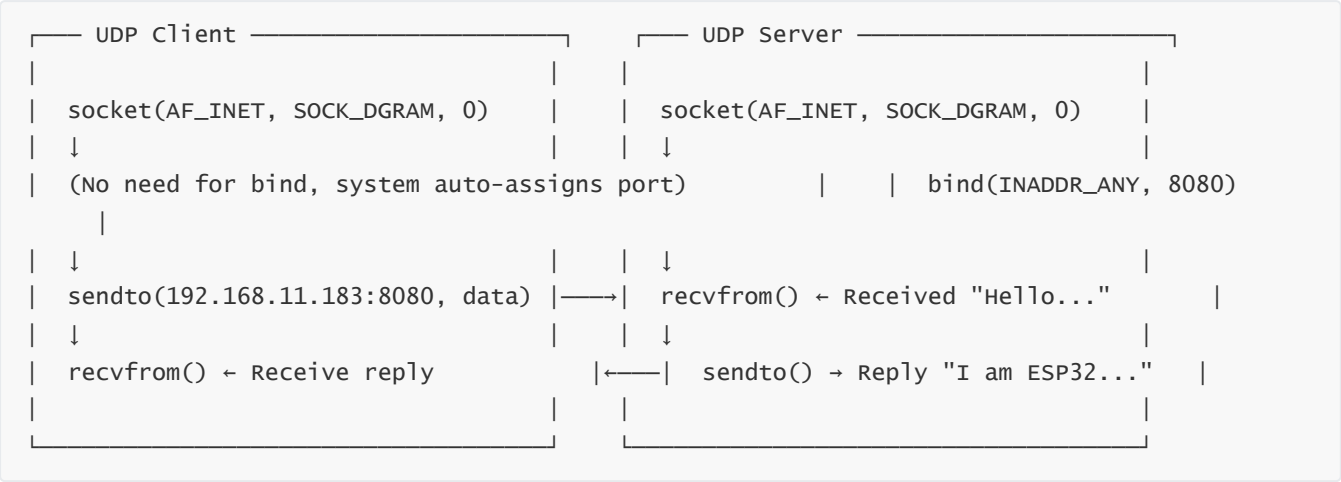
Feature	UDP	TCP
Connection	Connectionless, send directly	Three-way handshake to establish connection
Reliability	No guarantee of delivery, no guarantee of order	Guarantee delivery, guarantee order
Transmission Efficiency	High (8-byte header)	Lower (20-byte header)

Feature	UDP	TCP
Typical Use	Video streaming, DNS, IoT sensor data	Web, file transfer, MQTT
Core APIs	<code>sendto()</code> / <code>recvfrom()</code>	<code>send()</code> / <code>recv()</code>

This project contains **UDP client and server two sub-examples**: client sends data periodically and receives server replies; server listens on port, automatically replies to received data.

2. Core Concepts

2.1 UDP Socket Communication Flow



2.2 `sendto()` and `recvfrom()`

`sendto()` and `recvfrom()` are the two core APIs for UDP programming - the former sends datagrams with target address, the latter receives data while getting sender address for reply.

```
// Send: specify target IP and port
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);

// Receive: also get sender address (for reply)
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

2.3 Non-Blocking Receive (MSG_DONTWAIT)

`MSG_DONTWAIT` flag makes `recvfrom()` non-blocking - returns -1 immediately (errno=EAGAIN) when no data without blocking task. This example uses 10ms high-frequency polling, ensuring low-latency receive while not affecting timed sending:

```
int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                        // Non-blocking flag
                        MSG_DONTWAIT,
                        (struct sockaddr *)&from_addr, &len);
```

2.4 Network Byte Order

Network protocols use **Big-Endian** uniformly, while ESP32 is little-endian CPU. Ports and IP addresses must be converted - `htons()` converts port number, `inet_pton()` converts IP string to network-order binary:

```
// Port to network byte order
dest_addr.sin_port = htons(8080);
inet_pton(AF_INET, "192.168.1.1", &addr);
```

3. Project Architecture and Flow

3.1 UDP Client

```
app_main()
├─ wifi_init_sta()           ← WiFi STA initialization (wait for IP)
├─
├─ xTaskCreate(vudpClientTask, ...) ← Create UDP client task
├─ vudpClientTask():
│   ├─ socket(AF_INET, SOCK_DGRAM, 0) → Create UDP socket
│   ├─ Configure target address (192.168.11.183:8080)
│   └─ while(1):
│       ├─ recvfrom(MSG_DONTWAIT) ← Non-blocking receive reply (10ms polling)
│       ├─ If interval ≥ 2000ms → sendto() send data
│       └─ vTaskDelay(10ms)
```

4. Hardware Dependencies

- **Hardware Connection:** No external wiring, WiFi wireless communication
- **Client Dependent Components:** `lwip/sockets`, `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`, `FreeRTOS`
- **Server Dependent Components:** Same as above, additional `lwip/netdb`

5. Source Code Analysis — UDP Client

Complete source code: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `3.UDP` → `UDP-Client`

5.1 Macro Configuration

All adjustable parameters centralized at file top - WiFi credentials, target server address and port, timeout and send interval all managed via macros, no need to go inside functions to modify.

```
// WiFi SSID to connect
#define WIFI_SSID          "Your_WiFi_SSID"
#define WIFI_PASS          "Your_WiFi_Password" // WiFi password
#define UDP_SERVER_IP      "192.168.11.183"
#define UDP_SERVER_PORT    8080
#define WIFI_TIMEOUT_MS    15000 // WiFi timeout (ms)
#define UDP_SEND_INTERVAL_MS 2000
```

5.2 WiFi Initialization (Review)

This experiment's WiFi STA connection code (`wifi_init_sta()`, event callback, NVS initialization, etc.) is identical to **5.2 WIFI-STA-Mode**, not repeated here. Briefly: `app_main()` first `nvs_flash_init()` → then `wifi_init_sta()` wait for IP → finally `xTaskCreate()` create UDP task. WiFi connection is asynchronous, synchronized via EventGroup - only enter UDP send/receive logic after getting IP.

For details review: 5.Network Protocols and Wireless Communication->2.WIFI-STA-Mode->WIFI-STA-Mode, includes complete event-driven model, EventGroup synchronization mechanism and disconnect/reconnection implementation.

5.3 vUdpClientTask() — Send/Receive Loop

`vUdpClientTask()` approach is "receive first then send, don't interfere with each other" - each loop first uses non-blocking way to check if there's a reply, then sends data when it's time. Receive uses `MSG_DONTWAIT` (returns immediately if no data), send uses timestamp comparison (sends if 2 seconds have passed). Benefit is send and receive don't affect each other - unlike `vTaskDelay` where nothing can be done during delay.

```
static void vUdpClientTask(void *pv)
{
    int sock_fd;
    struct sockaddr_in dest_addr = {0}; // Destination address

    // Create UDP socket
    sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock_fd < 0) {
        ESP_LOGE(TAG, "Failed to create UDP socket");
        // Delete task on failure
        vTaskDelete(NULL);
    }

    dest_addr.sin_family = AF_INET; // IPv4
    // Port to network byte order
    dest_addr.sin_port = htons(UDP_SERVER_PORT);
    inet_pton(AF_INET, UDP_SERVER_IP, &dest_addr.sin_addr);

    TickType_t last_send_tick = 0; // Last send timestamp
    while (1)
```

```

{
    // Non-blocking receive
    char recv_buf[128] = {0};
    struct sockaddr_in from_addr;    // Sender address
    socklen_t len = sizeof(from_addr);

    int recv_len = recvfrom(sock_fd, recv_buf, sizeof(recv_buf)-1,
                           MSG_DONTWAIT,    // Non-blocking flag
                           (struct sockaddr *)&from_addr, &len);

    if (recv_len > 0) {
        // Add null terminator
        recv_buf[recv_len] = '\0';
        ESP_LOGI(TAG, "Reply: %s", recv_buf);
    }

    // Timed send (interval-triggered)
    TickType_t now = xTaskGetTickCount();
    if (now - last_send_tick >= pdMS_TO_TICKS(UDP_SEND_INTERVAL_MS)) {
        char send_buf[] = "Hello from ESP32 UDP Client!";
        int ret = sendto(sock_fd, send_buf, strlen(send_buf), 0,
                        (struct sockaddr *)&dest_addr, sizeof(dest_addr));
        if (ret > 0) {
            ESP_LOGI(TAG, "Sent: %s", send_buf);
        }
        last_send_tick = now;    // Update send timestamp
    }

    vTaskDelay(pdMS_TO_TICKS(10));
}
close(sock_fd);
vTaskDelete(NULL);
}

```

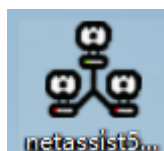
6. Operation Flow

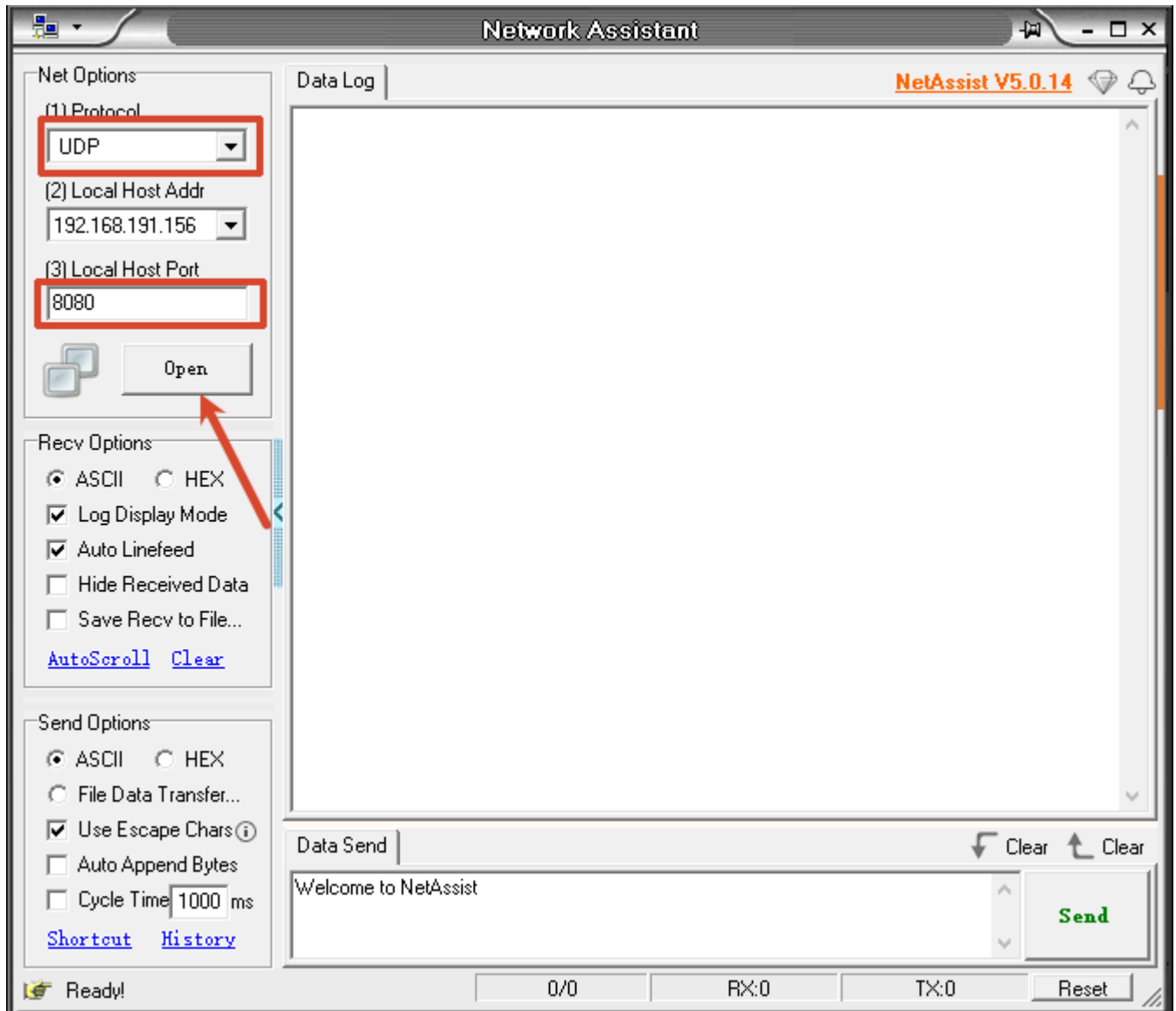
6.1 Compilation and Flashing Flow

ESP-IDF flashing flow: See [ESP-IDF → 1. Environment Setup → 3. Flashing Flow → Flashing Flow](#).

6.2 Test Method

1. Run network debug assistant on PC (such as NetAssist), select UDP protocol, listen on port 8080.





2. **Flash UDP_Client** to ESP32, serial will show send log, PC side should receive `Hello from ESP32 UDP Client!`.

```
[2026-07-11 17:32:59.077]# RECV ASCII/28 from 192.168.11.181 :64926 <<<
Hello from ESP32 UDP Client!
```

```
[2026-07-11 17:33:01.055]# RECV ASCII/28 from 192.168.11.181 :64926 <<<
Hello from ESP32 UDP Client!
```

3. PC side network debug assistant replies with anything, ESP32 serial will print `Reply: xxx`.

```
I (10173) UDP-CLIENT: Reply: Hello UDP Client
I (10483) UDP-CLIENT: Reply: Hello UDP Client
```

7.3 Normal Log — Client

```
I (2856) UDP-CLIENT: Got IP: 192.168.11.181
I (2858) UDP-CLIENT: WiFi connected successfully!
I (2860) UDP-CLIENT: UDP socket created successfully
I (2862) UDP-CLIENT: Sent: Hello from ESP32 UDP Client!
I (4862) UDP-CLIENT: Sent: Hello from ESP32 UDP Client!
```

```
I (2733) UDP-CLIENT: Got IP: 192.168.11.181
I (2733) UDP-CLIENT: WiFi connected successfully!
I (2733) UDP-CLIENT: UDP socket created successfully
I (2743) UDP-CLIENT: Sent: Hello from ESP32 UDP Client!
I (2743) main_task: Returned from app_main()
I (4743) UDP-CLIENT: Sent: Hello from ESP32 UDP Client!
I (6743) UDP-CLIENT: Sent: Hello from ESP32 UDP Client!
I (8743) UDP-CLIENT: Sent: Hello from ESP32 UDP Client!
I (10173) UDP-CLIENT: Reply: Hello UDP Client
I (10483) UDP-CLIENT: Reply: Hello UDP Client
```

8. Common Problems

Symptom	Cause	Solution
Client <code>Send failed</code> , <code>errno=xxx</code>	Target IP unreachable or port not listening	Check PC and ESP32 on same LAN, confirm PC UDP listening on 8080
Server can't receive client data	ESP32 IP changed / firewall blocked	Check ESP32 actual IP via serial, PC firewall allows 8080 port
High receive latency	<code>vTaskDelay</code> too large	Ensure polling interval $\leq 20\text{ms}$
<code>Failed to create UDP socket</code>	Socket descriptor exhausted	Check if forgot <code>close()</code> old Socket
<code>Failed to bind port</code>	Port already in use	Change <code>UDP_SERVER_PORT</code>
Continuous sends but PC only receives partial	UDP no flow control, network congestion packet loss	Increase send interval / add ACK confirmation on sender (custom application layer protocol)
Chinese text garbled	Terminal encoding inconsistency	Check serial assistant and network debug tool encoding settings (UTF-8)